

A Mini Project Report

On

SRMS Drona — A Bilingual Learning & Training Web Application for Non-Teaching Staff

In partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY in INFORMATION TECHNOLOGY

Submitted by

Anant Sharma (2400140130006)
Ansh Gupta (2400140130009)
Nafeha Aqueel (2400140130033)
Syeda Raazia (2400140130048)
Arsh Yar Khan (2300140130013)
Deepak Gangwar (2300140130018)
Riya Rastogi (2300140130043)
Rajneesh Kumar Singh (2300140130040)

Under Supervision of

Ms. Sakshi Goel

Department of Information Technology

Shri Ram Murti Smarak College of Engineering & Technology, Bareilly
Dr. A.P.J. Abdul Kalam Technical University, Lucknow

September, 2026

DECLARATION

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which, to a substantial extent, has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

All technical claims presented in this report have been verified against the running system and its public source repository. Performance figures quoted in Chapter 6 are the result of direct measurement of the deployed instance and are reproducible using the methods described therein.

Signature.....
Name.....
Roll No.....
Date.....
Signature.....
Name.....
Roll No.....
Date.....
Signature.....
Name.....
Roll No.....
Date.....
Signature.....
Name.....
Roll No.....
Date.....
Signature.....
Name.....
Roll No.....
Date.....
Signature.....
Name.....
Roll No.....
Date.....

CERTIFICATE

This is to certify that the Mini Project Report entitled “SRMS Drona — A Bilingual Learning & Training Web Application for Non-Teaching Staff”, which is submitted by Anant Sharma (2400140130006), Ansh Gupta (2400140130009), Nafeha Aqueel (2400140130033), Syeda Raazia (2400140130048), Arsh Yar Khan (2300140130013), Deepak Gangwar (2300140130018), Riya Rastogi (2300140130043) and Rajneesh Kumar Singh (2300140130040), is a record of the candidates' own work carried out by them under my supervision. The matter embodied in this work is original and has not been submitted for the award of any other work or degree.

ACKNOWLEDGEMENTS

We express our deepest gratitude to our mentors, advisors and colleagues who provided valuable guidance throughout the development of this project.

We are particularly thankful to Ms. Sakshi Goel for her technical insights, her review of successive drafts, and her guidance on structuring the requirements analysis and evaluation of the system.

We also thank Mr. Ashish Agarwal, Faculty Incharge of the Department of Information Technology, for providing the laboratory facilities required to develop and test this application, and the non-teaching staff who participated in user acceptance testing and whose feedback shaped several interface decisions recorded in Chapter 6.

Finally, we are grateful to our families and friends for their encouragement and patience throughout the completion of this work.

ABSTRACT

The SRMS Drona Learning Web Application is a web-based Learning Management System designed to digitize the internal training and professional development of non-teaching staff — office clerks, laboratory assistants and administrative personnel — at an educational institution. It replaces manual orientation sessions, verbal instruction and fragmented paper documentation with a single centralised platform.

The system provides staff with assigned courses structured as Category, Course, Module and Lesson; video lessons that resume from the point of departure; digital assessments with a 70% pass threshold and permitted retries; and instant generation of PDF certificates bearing a QR code that resolves to a public verification page. The interface is bilingual in English and Hindi across all 402 translatable strings, and the design is mobile-first for staff who access training during operational breaks.

For administrators the system provides a management console supporting course and lesson authoring, departmental and individual enrolment, an editable training calendar, staff roster import, an AI-assisted quiz generator, an HR analytics dashboard, a per-lesson watch-progress report with CSV export, and a certificate directory with search and filtering. A three-tier role hierarchy — staff, Head of Department, and Super Admin — governs access, with each Head of Department restricted to their own department.

The system is implemented in Django 6.0.8 on Python 3.12 with PostgreSQL, using server-rendered HTML, a custom CSS design system and 241 lines of vanilla JavaScript. A central design decision is that learning progress is derived server-side from accumulated watch time rather than trusted from the client, so completion cannot be forged by modifying browser requests. The application is verified by 203 automated tests and a dependency audit reporting zero known vulnerabilities, and is deployed on Render with continuous integration through GitHub Actions. Three real security vulnerabilities were identified during development by testing each control against its attacker; all are documented in this report with their root causes, detection method and remediation.

Table of Contents

DECLARATION

CERTIFICATE

ACKNOWLEDGEMENTS

ABSTRACT

CHAPTER 1

INTRODUCTION

1.1

Background and Motivation

1.2

Problem Statement

1.3

Objectives

1.4

Scope of the Project

CHAPTER 2

LITERATURE REVIEW

2.1

Existing Learning Management Systems

2.2

Limitations of Traditional Institutional Training

2.3

Comparative Analysis

2.4

Advantages of a Purpose-Built System

CHAPTER 3

SYSTEM REQUIREMENTS AND ANALYSIS

3.1

Functional Requirements

3.2

Non-Functional Requirements

3.3

Use Case Diagram

3.4

Data Flow and System Interactions

CHAPTER 4

SYSTEM DEVELOPMENT METHODOLOGY

4.1

Development Methodology

4.2

Tools and Technologies

4.3

System Design Technique

4.4

Authentication and Access Control Technique

4.5

Progress Tracking Technique

4.6

Media Storage and Streaming Technique

4.7

Bilingual Interface Technique

4.8

Certificate Generation and Verification Technique

4.9

AI-Assisted Quiz Generation Technique

4.10

Verification Technique

4.11

Security Engineering Technique

4.12

Deployment Technique

CHAPTER 5

MODULES

5.1

User and Authentication Module

5.2

Course and Content Module

5.3

Quiz and Assessment Module

5.4

Certificate Module

5.5

Analytics and Reporting Module

5.6

Management Console Module

5.7

Notification Module

CHAPTER 6	
RESULTS AND DISCUSSION	
6.1	
Functional Verification	
6.2	
Performance Measurement	
6.3	
User Feedback	
6.4	
Vulnerabilities Identified and Remediated	
6.5	
Challenges Encountered and Resolutions	
6.6	
Limitations	
CHAPTER 7	
CONCLUSION AND FUTURE SCOPE	
CHAPTER 8	
REFERENCES	
APPENDIX A	
Source Repository	

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Non-teaching staff — office clerks, laboratory assistants, library staff and administrative personnel — form the operational backbone of an educational institution. Their competence in laboratory safety procedures, administrative regulations, document handling and workplace conduct directly determines the quality of service experienced by students and teaching staff. Continuous professional development for this group is therefore not optional; it is a precondition of reliable institutional functioning.

The conventional method of achieving this in Indian institutions is the orientation session: a scheduled, in-person meeting in which policies are explained verbally, documents are distributed on paper, and attendance is recorded in a register. This approach carries well-documented weaknesses. Knowledge transfer is inconsistent, because the quality of a session depends entirely on the individual delivering it and on who happened to attend. Materials are scattered across physical documents, notice boards and informal verbal instruction, so a member of staff who missed one session has no route to the content. Records are maintained manually, which means that verifying whether a given employee completed a given module requires an act of archival retrieval. And because the sessions occupy working hours, they compete directly with the daily operational duties of the very staff they are intended to serve.

A second and equally important consideration is linguistic. The intended users of an institutional training system are precisely those least likely to have encountered English-medium professional software in the course of their work. Commercial learning platforms, however sophisticated, are designed for a general market and are available predominantly in English with an interface vocabulary — “enrol”, “module”, “submission” — that carries no meaning in an institutional context for a Hindi-speaking clerk. The result is a paradox: the staff with the greatest need for structured training are those least able to use the tools that provide it.

The SRMS Drona Learning Web Application addresses these gaps directly. It provides a centralised, mobile-first, bilingual platform in which training content is authored once by administrators and consumed by staff on any device, in either English or Hindi, with every stage of completion recorded and verifiable. The remainder of this chapter formalises the problem, the objectives derived from it, and the boundaries of the delivered system.

1.2 Problem Statement

The current staff training process is manual and fragmented. Orientation sessions are scheduled infrequently and clash with roster duties; there is no mechanism for a member of staff to access material they missed; and completion is evidenced only by a signature in a register. Administrators, meanwhile, maintain no consolidated view of who has completed what, so questions of the form “did this employee finish the safety module?” cannot be answered without manual inspection.

Three consequences follow. For staff, learning is episodic rather than continuous, and there is no way to resume interrupted content. For administrators, the absence of centralised records produces both administrative overhead and the risk of inaccurate certification. For the institution as a whole, there is no defensible evidence of training completion when an audit, accreditation visit or incident investigation requires it.

There is therefore a clear requirement for a web-based Learning Management System, designed specifically for non-teaching staff, that delivers structured video and document modules, records

progress reliably, assesses comprehension digitally, issues verifiable certificates, and gives administrators real-time visibility across departments — while remaining accessible to users with limited technical familiarity through a bilingual, mobile-first interface.

1.3 Objectives

1. To deliver a centralised web platform through which non-teaching staff can access assigned training courses at any time and on any device.
2. To author training content in a four-level hierarchy — Category, Course, Module and Lesson — supporting both video and document material.
3. To record learning progress reliably in a manner that cannot be falsified by the learner, and to resume interrupted lessons from the point of departure.
4. To provide digital assessment with a defined pass threshold, permitted retries, and immediate feedback.
5. To generate verifiable PDF certificates automatically on completion, carrying a QR code that resolves to a public verification page.
6. To enforce a three-tier role hierarchy — staff, Head of Department, Super Admin — so that each administrative act is confined to its proper scope.
7. To provide administrators with a management console for course authoring, enrolment, scheduling, and staff account administration.
8. To provide an AI-assisted quiz generator that converts an uploaded SOP document into assessment questions.
9. To provide HR analytics and a per-lesson watch-progress report with CSV export for evidence-based oversight.
10. To deliver a fully bilingual interface in English and Hindi, covering navigation, forms, validation messages and error pages.
11. To verify the system through an automated test suite and a dependency security audit, and to deploy it with continuous integration.

1.4 Scope of the Project

The delivered system comprises the following:

- A responsive web application covering authentication, course consumption, assessment, certification and administration.
- A relational database storing employee profiles, course hierarchies, lesson media, progress records, assessment attempts and certificate metadata.
- Auto-saved, server-derived learning progress with resume support.
- Digital quizzes with a 70% pass threshold, retries, and an AI generation path with disclosed offline fallback.
- Automated certificate generation with QR-based public verification and surname masking.
- A management console supporting course, module and lesson authoring, departmental and individual enrolment, an editable training calendar, and staff roster import.
- A three-tier role hierarchy with department-scoped administrative authority.
- An HR analytics dashboard and a per-lesson watch-progress report, both with CSV export.

- A bilingual interface implemented across 402 translatable strings and paired bilingual content fields.
- Automated deployment to a cloud platform with continuous integration and dependency auditing.

The following are explicitly outside the scope of this project and are recorded as future work in Chapter 7: external email delivery, which requires SMTP credentials not held by the institution; a native mobile application; integration with third-party learning content providers; and multi-institution federation.

Two design decisions deserve early statement because they shape much of what follows. First, a third-party single-sign-on integration present in an early iteration was removed; the reasoning is given in Section 4.4.1. Second, the system stores lesson video in the database rather than in object storage, a deliberate trade-off analysed in Section 4.6.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the existing solutions available for institutional training, identifies the specific limitations that justify building a dedicated system, and establishes the comparative basis for the design decisions recorded in Chapter 4.

2.1 Existing Learning Management Systems

2.1.1 Moodle

Moodle is the dominant open-source learning management system, and it is the most credible general-purpose reference for this class of software. It provides a comprehensive course-authoring model, a question bank supporting multiple assessment types, a gradebook, and completion-tracking certificates. Its certificate feature can generate a PDF carrying a QR code, which is the approach adopted in this project.

Its limitations with respect to the present requirements are threefold. Administration requires a substantial technical investment: an administrator must configure roles, capabilities, enrolment methods and plugin behaviour, none of which is self-evident. The interface is dense and oriented toward teachers and learners in formal education, presenting a significant adoption barrier for the non-technical administrative staff this project targets. And its data model presupposes a learner–teacher relationship, an assumption that does not hold for a compliance-oriented staff training programme in which there is no teacher and assessment exists only to confirm comprehension.

2.1.2 Commercial Platforms

Commercial platforms such as Udemy and Coursera offer excellent self-paced video learning with embedded assessment, and their video player behaviour informed the streaming technique adopted in Section 4.6. They are however fundamentally unsuited to this use case. Their unit of value is a public marketplace course sold to an individual, not a mandatory compliance module assigned by an institution to a specific department. They provide no mechanism for an administrator to assign training to identified employees, to verify that a named employee completed it, or to extract a per-employee completion record for audit. Their content is English-language and lacks Hindi localisation.

2.1.3 Custom Institutional Systems

The third category is bespoke institutional systems, which can be tailored exactly to local requirements and which therefore represent the realistic alternative to this project. The trade-off is maintenance: such systems carry the full cost of authentication, role management, media serving, backup and security maintenance, with no vendor responsible for any of it. The literature is clear that bespoke institutional systems succeed where the institution already possesses the engineering capacity to own the result, and fail where that capacity is absent. This project is an instance of the former: it is delivered inside a Department of Information Technology, and the design decisions in Chapter 4 were made on the assumption of continuing departmental ownership.

2.2 Limitations of Traditional Institutional Training

The manual baseline against which this system is measured exhibits five specific limitations, each of which is directly addressed by a feature of the delivered system.

1. Scheduling conflict. In-person sessions occupy working hours and collide with roster duties. Addressed by asynchronous, on-demand delivery.
2. Inconsistent knowledge transfer. Quality depends on the individual delivering the session and on attendance. Addressed by centrally authored content with a single authoritative version.
3. No progress oversight. Administrators cannot determine who completed what without manual record inspection. Addressed by per-employee, per-lesson progress records surfaced in a filterable report.
4. Manual verification. Paper certificates are slow to issue, error-prone to record, and impossible to verify externally. Addressed by automatic PDF generation with QR-based public verification.
5. Language and usability barriers. Generic external content is English-only and assumes technical familiarity. Addressed by a bilingual interface and a mobile-first design.

2.3 Comparative Analysis

Availability
 Scheduled in-person sessions
 On-demand
 On-demand, mobile-first
 Content structure
 Physical documents, verbal instruction
 Fixed or marketplace course structure
 Category → Course → Module → Lesson
 Media handling
 None
 External or hosted video
 Video stored with HTTP Range streaming
 Progress tracking
 Paper register
 Generalised analytics
 Per-employee, per-lesson watch time
 Progress integrity
 Not applicable
 Client-reported in many configurations
 Server-derived, non-forgable
 Assessment
 Manual / paper quiz
 Configurable question bank
 MCQ with 70% threshold and retries
 Question authoring
 Manual
 Manual or plugin-dependent
 AI-assisted from SOP document, with disclosed fallback
 Certification
 Physical paper certificate
 Digital certificate
 PDF with QR, public verification, surname masked
 Administrative scope
 Single administrator
 Global administrator

Three tiers; HOD scoped to department
 Language support
 None
 Primarily English
 English and Hindi, 402 strings plus content fields
 Content language
 As delivered
 As delivered
 Paired bilingual content fields
 Reporting
 None
 General reports
 HR dashboard, watch-progress grid, CSV export
 Verification method
 Signature inspection
 Platform record
 Public QR scan, no login required

Table 2.1 — Capability comparison across the four approaches.

2.4 Advantages of the Delivered System

Efficiency. Automated progress recording, assessment scoring and certificate generation remove the manual record-keeping that dominates the conventional process.

Defensible records. Because progress is derived server-side and certificates carry a scannable verification code, completion can be evidenced to a third party without granting access to the system.

Accessibility. On-demand mobile-first delivery permits staff to complete training during operational breaks, and bilingual support removes the language barrier that limits adoption of English-only platforms.

Targeted oversight. Department-scoped authority means a Head of Department sees their own department without access to others, and the Super Admin holds a platform-wide view.

Honest degradation. Where an automated feature cannot operate — most significantly AI question generation when no API key is configured — the system falls back to a defined alternative and states plainly to the administrator that fallback occurred, rather than presenting it as the automated result.

CHAPTER 3

SYSTEM REQUIREMENTS AND ANALYSIS

This chapter specifies what the system must do and the qualities it must possess, identifies the actors and their interactions, and traces the principal data flows. Requirements are stated here in technology-neutral terms; the mechanisms satisfying them are presented in Chapter 4.

3.1 Functional Requirements

FR-01

A user shall authenticate using an institutional employee identifier and password.

Section 4.4.1

FR-02

The system shall distinguish three roles: staff, Head of Department, and Super Admin.

Section 4.4.2

FR-03

A user shall be able to register, and the account shall remain inactive until approved by an administrator.

Section 5.1

FR-04

A manager shall be able to create, edit and delete courses, categories, modules and lessons.

Section 5.6

FR-05

An administrator shall be able to enrol a whole department in a course, or a single named employee.

Section 5.6

FR-06

A manager shall be able to schedule, edit and delete training sessions on a calendar.

Section 5.6

FR-07

The system shall record watch time per employee per lesson and derive completion from it.

Section 4.5

FR-08

A lesson shall resume from the last recorded position.

Section 4.5

FR-09

Video shall be seekable within the player.

Section 4.6

FR-10

A user shall attempt a quiz and receive an immediate score against a 70% threshold.

Section 5.3

FR-11

A failed attempt shall be permitted to be retried.

Section 5.3

FR-12

An administrator shall be able to generate a quiz from an uploaded SOP document.

Section 4.9

FR-13

The system shall issue a PDF certificate automatically on successful completion.

Section 4.8

FR-14

A certificate shall carry a QR code resolving to a public verification page.

Section 4.8

FR-15

The verification page shall not disclose the holder's surname.
 Section 4.8
 FR-16
 An administrator shall be able to search and filter a certificate directory.
 Section 5.4
 FR-17
 A Head of Department shall see only their own department's data.
 Section 4.4.2
 FR-18
 A Super Admin shall hold platform-wide visibility.
 Section 4.4.2
 FR-19
 The system shall display an HR analytics dashboard.
 Section 5.5
 FR-20
 The system shall display a per-lesson watch-progress report and export it as CSV.
 Section 5.5
 FR-21
 The entire interface shall be available in English and Hindi, switchable by the user.
 Section 4.7
 FR-22
 Course, module, lesson and quiz content shall be authorable in both languages.
 Section 4.7
 FR-23
 An administrator shall be able to import a staff roster.
 Section 5.6
 FR-24
 The system shall record administrative actions in an auditable log.
 Section 5.6
 FR-25
 The system shall display a customised error page for 403, 404 and 500 conditions.
 Section 4.11

Table 3.1 — Functional requirements and the sections that satisfy them.

3.2 Non-Functional Requirements

NFR-01
 Security
 Role-based access control; no credential in source; production refuses to boot on default secrets
 Section 4.11, 203-test suite
 NFR-02
 Progress integrity
 Completion derivable only from server-held watch time
 Named regression tests
 NFR-03
 Usability
 Usable by non-technical staff with no training; mobile-first
 Chapter 6 UAT
 NFR-04
 Localisation
 Full interface in English and Hindi
 HindiCatalogueTests, 402 strings
 NFR-05

Maintainability
 Modular app-per-domain structure; 203 automated tests
 Section 3.3, 4.10
 NFR-06
 Data integrity
 Relational constraints; server-side derived state
 Section 4.3.1
 NFR-07
 Portability
 Runs locally with SQLite; deploys with PostgreSQL
 Quick-start procedure
 NFR-08
 Availability
 Automated deployment; health monitoring
 Section 4.12
 NFR-09
 Dependency hygiene
 No known vulnerable dependency
 pip-audit: 0 advisories
 NFR-10
 Performance
 Interactive page response within a usable interval
 Section 6.2 measurements

Table 3.2 — Non-functional requirements with their verification method.

3.3 Use Case Diagram

The system has three external actors, distinguished by the authority each holds rather than by a separate login mechanism.

Actors

- Non-Teaching Staff (Employee) — authenticates, browses assigned courses, watches video lessons, reads SOP documents, attempts quizzes, tracks personal progress, and downloads own certificates.
- Head of Department (HOD) — holds all staff capabilities for their own department, plus course authoring, departmental and individual enrolment, session scheduling, staff import, HR analytics and the watch-progress report.
- Super Admin — holds all HOD capabilities, plus staff and HOD account creation, signup approval, platform-wide analytics and the full certificate directory.
- External Verifier — unauthenticated; scans a certificate QR code to confirm authenticity.

Use Cases by Actor

Head of Department

All staff use cases, plus: Author course / module / lesson · Enrol department · Assign individual employee · Schedule training session · Import staff roster · View HR dashboard · View watch-progress report · Export CSV · Approve signup · Generate quiz from SOP

Super Admin

All HOD use cases, plus: Create staff / HOD account · View all-department analytics · View full certificate directory · Search and filter certificates

External Verifier

Verify certificate authenticity via QR code (no authentication)

Table 3.3 — Use cases by actor. A text use-case diagram is provided here; the corresponding screen set is documented in the source repository.

3.4 Data Flow and System Interactions

3.4.1 Staff Interaction Flow

1. The user submits an employee identifier and password at the login page.
2. The system verifies the credential against the stored Argon2 hash, subject to per-IP rate limiting.
3. On success the user is routed to a role-appropriate dashboard showing enrolled courses and their progress.
4. The user selects a course, which presents its modules and lessons with per-lesson completion state.
5. Selecting a video lesson loads the player; the client requests the media with a byte range.
6. During playback the client emits a periodic heartbeat carrying elapsed seconds.
7. The system clamps the interval, accumulates the total against the lesson duration, and derives completion at 90%.
8. The user attempts the quiz; the system scores the submission against the 70% threshold.
9. On passing, a certificate is generated, stored, and offered for download.

3.4.2 Administrator Interaction Flow

1. The administrator authenticates and enters the management console.
2. They create a category and course, then add modules and lessons, uploading video or document material.
3. They optionally upload an SOP document and generate assessment questions from it.
4. They enrol a department, or assign named individuals, and schedule training sessions.
5. The system records each administrative action in the audit log.
6. The administrator monitors completion through the HR dashboard and the watch-progress report, and exports either as CSV.

3.4.3 System and Automated Flow

Two background activities operate independently of user requests. A scheduled job runs at a configured interval, identifies staff with pending training, and prepares reminder messages; delivery is currently directed to the application log, and the system records a warning that delivery is not configured rather than reporting a silent success. Separately, a maintenance command prunes audit-log entries beyond a 180-day retention window.

CHAPTER 4

SYSTEM DEVELOPMENT METHODOLOGY

This chapter presents the methodology adopted and the specific techniques applied. Each technique is stated with its rationale and, where relevant, the source file that implements it, so that every claim is independently verifiable against the project's repository.

4.1 Development Methodology

The system was developed using an iterative-incremental methodology, structured as a sequence of working vertical slices rather than a phased waterfall. Each iteration delivered a capability that ran end-to-end against a real database, and no slice was considered complete until covered by an automated test.

This approach was selected over the waterfall for three reasons established during requirements analysis. First, the requirements most likely to be misinterpreted were the access-control rules, and these can only be validated by exercising them against real requests rather than by reviewing them on paper. Second, the project carried an external dependency — the AI quiz service — whose behaviour could not be assumed without testing, so an incremental approach permitted a working offline fallback to be established first and the integration added second. Third, iterative delivery produced a continuously deployable system, which allowed the security weaknesses described in Section 6.4 to be discovered and remediated against a running instance rather than theorised about.

The methodology comprised four recurring activities: requirement elicitation from target users; analysis and modelling of roles, data and flows; incremental implementation; and continuous verification by automated test.

4.2 Tools and Technologies

Backend framework

Django 6.0.8 (Python 3.12)

Supplies authentication, ORM, migrations, internationalisation, admin interface and CSRF/XSS protections as framework features, so these controls are inherited rather than reimplemented.

Database

PostgreSQL (production); SQLite (development and tests)

The relational model suits the heavily-referenced content hierarchy. SQLite allows the full suite to run with no external service.

Frontend

Server-rendered HTML; custom CSS design system; vanilla JavaScript (241 lines)

Removes a client build step and a framework runtime, producing pages that load without JavaScript — appropriate for users with limited technical familiarity.

Password hashing

Argon2 via `argon2-cffi`

Memory-hard algorithm, positioned first in `PASSWORD_HASHERS` (`srms_dorna/settings.py:121`).

AI integration

Google Gemini (`google-genai`)

Generates assessment questions from SOP documents, with a disclosed rule-based fallback.

PDF and QR

ReportLab and `qrcode`

Renders certificates and embeds a verification code.

Media storage

Custom Django Storage backend
 Stores lesson media in the database, removing a dependency on object storage (analysed in 4.6).
 Scheduling
 APScheduler
 Single-worker training reminder loop.
 Version control and CI
 Git and GitHub Actions
 Automated pipeline runs the complete test suite on every push and pull request.
 Deployment
 Render with external PostgreSQL
 Cloud hosting with a free tier suitable for an institutional prototype.

Table 4.1 — Technology selection and rationale.

The resulting codebase comprises approximately 5,159 lines of application Python across seven domain applications, 2,100 lines of tests, 38 HTML templates, and 2,019 lines of CSS and JavaScript.

4.3 System Design Technique

The system was designed as a monolithic Django application with strict app-per-domain decomposition. The codebase is partitioned into seven applications — users, courses, quizzes, certificates, analytics, management and notifications — each owning its models, views, tests and URL namespace.

A modular monolith was chosen over microservices deliberately. The workload is a small number of concurrent users reading and writing a shared relational dataset. Distributed services would have introduced network failure modes and cross-service transaction coordination for no performance benefit at this scale, and the certificate-generation workflow specifically requires atomicity across progress, assessment and credential records. App-per-domain decomposition captures the essential modularity while preserving single-transaction integrity.

4.3.1 Data Model Design

The data model derives from the requirement that learning progress be attributable to an identified individual and defensible under audit. The entity structure is:

- Category → Course → Module → Lesson — a four-level hierarchy, with a ManyToMany relation from Course to target Department enabling department-scoped mandatory courses.
- StaffUser — a custom user model declaring USERNAME_FIELD = 'employee_id' (apps/users/models.py:46), so authentication uses the institutional employee number rather than an email address. It carries department, role, preferred_language, phone_number and designation.
- Department — name, unique code and description, the unit of administrative scoping.
- Enrollment — links a staff member to a course, holding an aggregate progress_percent.
- LessonProgress — per-employee, per-lesson record declaring unique_together = ('enrollment', 'lesson'), so duplicate progress rows are impossible at the database level rather than only in application code.
- StoredUpload — binary storage for lesson video, SOP documents and generated certificate PDFs.
- Quiz, Question, Choice, QuizAttempt — assessment entities; Quiz.passing_score defaults to 70, and QuizAttempt records score, total_questions and passed.
- Certificate — a uniquely identified credential linked to both the staff member and the course, with a generated identifier and an issued timestamp.

Referential integrity is enforced by foreign keys with policies chosen per relationship. LessonProgress cascades from Enrollment, so removing an enrolment removes its progress records. TrainingSession.course uses on_delete=SET_NULL, so deleting a course from the calendar preserves the scheduled event record — appropriate because scheduled sessions are historically meaningful. Certificate cascades from both the user and the course.

4.3.2 Architectural Layers

The deployed system comprises four logical layers, described in Section 5 and evidenced by the screen set in the source repository: a presentation layer of 38 server-rendered templates; an application layer of domain applications encapsulating business logic; a data layer of PostgreSQL accessed through the Django ORM; and an integration layer comprising the AI service, the PDF/QR generator and the reminder scheduler.

4.4 Authentication and Access Control Technique

4.4.1 Authentication

Authentication uses Django's ModelBackend with server-side sessions, and passwords are hashed with Argon2.

A significant decision was the removal of a third-party single-sign-on integration (Clerk) present in an early iteration. Testing exposed two production defects in that integration. The publishable key was being supplied using the npm/ESM calling convention to the CDN global build, which silently rendered an empty authentication widget even though the key was present and correctly rendered into the page. Separately, the feature gate tested the secret key but never the publishable key, so a missing key produced a visible but non-functional button rather than a clean disablement. Because an external identity provider introduces a permanent third-party trust boundary and runtime dependency, and because removal reduced the login page to a single form, the system was simplified to employee identifier and password. Both defects are documented in the project engineering notes so that any future reintroduction is a considered decision rather than an inadvertent reversal.

4.4.2 Role-Based Access Control

staff

Access to assigned courses, video lessons, SOP documents, quizzes, own progress, own certificates, training calendar, language preference.

hod

All staff capabilities, plus: course and lesson authoring, departmental and individual enrolment, session scheduling, staff import, signup approval, HR analytics and watch-progress report — scoped to their own department.

admin

All HOD capabilities without departmental restriction, plus: staff and HOD account creation, platform-wide analytics, and the full certificate directory.

Table 4.2 — Role authority matrix.

Two structural techniques make the authorisation trustworthy rather than merely present.

Single source of truth for the manager check

Rather than testing role membership at each call site, a single is_manager property is defined on the user model (apps/users/models.py:50) and every management view consults it. Divergent inline role

checks are the characteristic cause of authorisation bypass; centralising the predicate eliminates the class of defect rather than the individual instance.

Single access gate for course content

The rule “a manager may preview any course; any other user must be enrolled” was originally implemented in three separate places with drifting behaviour. It was centralised into a `can_view_course()` predicate in `apps/courses/access.py`. A deliberate design note accompanies this: only the predicate is shared, because each caller retains its own failure response. A quiz denial returns a message and redirects, whereas a video denial returns 404 so that a probe cannot distinguish “not enrolled” from “does not exist”. The rule and its differing failure modes are therefore both explicit and centralised.

Anti-enumeration

The login view passes through Django's `authenticate()`, which computes a hash against a dummy value when the identifier is unknown. The error message and the response timing are therefore identical whether or not the account exists. A named regression test, `test_unknown_employee_id_still_runs_the_password_hasher`, fails if this behaviour is ever lost.

4.5 Progress Tracking Technique — Server-Derived Completion

This is the central technical technique in the system, and it exists to satisfy one requirement: learning progress must not be forgeable by the learner.

The naive design — the player posts `completed: true` when it reaches the end — is defeated by a single crafted HTTP request. The implemented technique derives completion entirely from watch time accumulated on the server. The client is a transport mechanism only; it is never trusted to assert a completion state.

Client behaviour

The video element emits a heartbeat at approximately ten-second intervals carrying the elapsed interval. This reports elapsed time, nothing more.

Server-side accumulation

The `register_watch()` method in `apps/courses/models.py` applies three constraints in sequence.

1. Per-heartbeat clamp. The incoming interval is clamped to `HEARTBEAT_MAX_SECONDS = 30`. A request claiming 3,600 seconds is credited 30. This alone prevents a single request from completing a long lesson.
2. Running-total clamp. The accumulated total is capped at the lesson's own duration, so repeated heartbeats cannot inflate progress beyond one complete viewing. The method returns only the number of seconds actually credited, which becomes zero once the cap is reached.
3. Threshold derivation. Completion is set when the accumulated total reaches $\text{COMPLETION_RATIO} \times \text{lesson duration}$, where $\text{COMPLETION_RATIO} = 0.9$.

The 0.9 threshold is a deliberate engineering choice rather than a rounded convenience. Because heartbeats arrive in roughly ten-second steps, a threshold of 1.0 would leave a fully and legitimately watched lesson at approximately 97%, permanently blocking its certificate. The margin absorbs the quantisation of the heartbeat without materially permitting a partial watch to pass.

Non-video lessons

PDF documents and lessons with no recorded duration cannot be watch-verified. Rather than auto-completing on a zero duration — which would award completion for an unopened document — the model exposes a `requires_watch_time` property, and such lessons require an explicit acknowledgement. The acknowledgement is recorded distinctly from a derived completion.

Resume support

Because the last position is persisted with each heartbeat, an interrupted lesson resumes from the point of departure rather than from the beginning.

Verification

Two named regression tests enforce the technique: `test_forged_completion_flag_is_ignored_for_video` asserts that a posted completion flag earns nothing, and `test_single_heartbeat_cannot_credit_whole_video` asserts that a single request cannot claim an entire lesson.

4.6 Media Storage and Streaming Technique

4.6.1 Storage in the database

Lesson video, SOP documents and generated certificate PDFs are stored in the database through a custom Django Storage subclass rather than on the filesystem or in object storage.

This is an explicit trade-off. Storing media in the database removes an entire class of backup and consistency problem and reduces the deployment to a single managed service, since a database backup then covers application data and media together. The costs are database growth — the deployed instance holds roughly 54 MB of course media — and memory cost during serving, since the view must hold the bytes to slice them.

4.6.2 Upload validation on two independent signals

Validation requires the declared Content-Type and the filename extension to agree, after which the stored type is re-derived from the extension using Python's `mimetypes` module (`apps/courses/storage.py`). The declared type is attacker-controlled input and is never what is persisted or served.

This dual-signal technique was adopted in direct response to a stored cross-site scripting vulnerability, described in Section 6.4. Size caps are enforced in the storage layer: 8 MB for documents and 25 MB for video, held in separate storage classes so the two policies cannot drift.

4.6.3 HTTP Range support

A video element requests byte ranges to scrub within a clip, and several browsers refuse to play at all if the server ignores the Range header and replies with 200. The `_range_response()` helper in `apps/courses/views.py` parses the header and handles both explicit ranges (`bytes=0-1023`) and suffix ranges (`bytes=-500`), returning 206 Partial Content with a correct Content-Range header for valid ranges and 416 Range Not Satisfiable with Content-Range: `bytes */<total>` when the requested start lies beyond the file. Partial responses are streamed rather than buffered, so server memory does not scale with file size.

Video responses are additionally gated on enrolment, returning 404 rather than 403 so that an unauthorised probe cannot distinguish “not enrolled” from “does not exist”.

4.7 Bilingual Interface Technique

Bilingual support was implemented through Django's native internationalisation framework rather than ad-hoc conditional rendering, so that English and Hindi are peers rather than one language with a translation overlay.

Interface strings

402 translatable strings are catalogued in `locale/hi/LC_MESSAGES/django.po`, covering navigation, buttons, form labels, validation messages, email subjects, the management console and every error page. Language selection resolves in a defined order — session preference first, then the user's saved `preferred_language` — implemented in `UserLanguageMiddleware`.

Bilingual content

Course, module, lesson and quiz titles and descriptions are stored in paired columns (`title` and `title_hi`, `text` and `text_hi`) and selected in templates according to the active language, falling back to English where a translation has not been supplied. This is treated separately from interface translation because course content is data rather than interface text; an administrator entering a Hindi lesson title affects content, not the application chrome.

Compiled catalogue as a build artefact

The compiled binary catalogue is committed to the repository. This is a deployment requirement rather than a convenience: the production Python runtime provides no message-compilation utility, so a build-time compilation step would fail or be silently skipped, leaving the deployed instance without Hindi strings. A dedicated test class, `HindiCatalogueTests`, fails if the catalogue is absent, if a known string stops translating, or if the language preference stops reaching the template layer.

Categories never translated

Two categories are documented as never translatable, because translating them caused real breakage. A language code embedded in a URL must not be translated, since the translated word would be submitted as the code and silently break the toggle. And values that are data rather than display text — employee identifiers, department codes, certificate identifiers — must not be translated.

4.8 Certificate Generation and Verification Technique

Certificates are generated server-side as PDF documents using `ReportLab`, with a QR code embedded by the `qrcode` library. The QR resolves to a public verification URL constructed as `/verify/<certificate_id>/` (`apps/certificates/pdf_builder.py`).

Identity masking is a deliberate privacy technique. The public verification page returns a display name derived from the first name only, via `_masked_name()` in `apps/certificates/views.py`. This allows any third party — an employer, an auditor, a verifying officer — to confirm that a certificate is genuine without the page becoming a directory of staff names and their qualifications. The certificate identifier itself is unguessable.

Access control on the artefact is enforced per file rather than by directory. The certificate PDF is served through a protected media route, not a public static path: a user may retrieve their own certificate, or a certificate belonging to a member of staff they manage. A filename guess returns 404. The named regression test `test_other_staff_cannot_fetch_someone_elses_certificate` guards this property.

4.9 AI-Assisted Quiz Generation Technique

Assessment generation uses the Google Gemini API to convert a Standard Operating Procedure document — supplied as an uploaded PDF or as pasted text — into multiple-choice questions with answer keys and explanations (apps/quizzes/gemini_services.py).

Explicit model ordering

Rather than hardcoding a single model identifier, the service holds an ordered tuple of candidate models and is pinned at runtime through an environment variable. A hardcoded model list silently degrades as preview models are retired by the provider, producing a failure whose cause is not obvious from the error. Ordering the tuple with stable models first, and allowing a pinned override, makes the active configuration visible in deployment settings rather than buried in source.

Honest degradation

When no API key is configured, or every candidate model fails, the service falls back to a rule-based generator. The essential technique here is that the fallback is disclosed: a warning is logged and the administrator is informed in the interface that the questions were not AI-generated.

Presenting template questions as artificial-intelligence output would misrepresent the system's capability to the person relying on it, and in a compliance-training context an administrator may reasonably assume generated questions have been reviewed for correctness. Two named regression tests enforce the disclosure: `test_missing_api_key_is_logged_not_silent` fails if the warning is removed, and `test_fallback_honours_the_requested_count` ensures the fallback does not silently return a different number of questions than requested.

4.10 Verification Technique

4.10.1 Automated testing

The system is verified by an automated suite of 203 tests executed on every push and pull request, distributed as follows.

users	
54	Authentication, RBAC, signup approval, rate limiting, anti-enumeration, profile management
31	Management console, authorisation gating, audit log
courses	
23	Content hierarchy, progress derivation, media storage and streaming
quizzes	
21	Assessment scoring, retries, AI generator and fallback disclosure
analytics	
20	Dashboards, watch-progress report, CSV export, manager scoping
certificates	
17	Generation, QR embedding, public verification, surname masking
notifications	
1	Scheduler configuration and warning behaviour

Total
203

Table 4.3 — Test distribution by application.

Testing runs against a dedicated test settings module that forces an in-memory database, disables the scheduler, clears the AI API key so AI tests exercise the offline path, substitutes an in-memory cache backend because an in-memory database cannot host the production database cache, and uses a fast password hasher. Generated PDFs are written to a temporary media root, so a test run leaves the working tree clean and cannot corrupt development data.

4.10.2 Defect-directed testing

Beyond coverage of specified behaviour, the technique of testing each implemented control against its attacker was applied systematically. Three vulnerabilities were found this way, all of which had passed the then-existing test suite and were configured correctly. Each finding was converted into a named regression test, so the suite grew to include defences rather than only features. Section 6.4 presents these findings in full.

4.10.3 Dependency audit

A software composition analysis is run against the fully pinned dependency set. It initially reported 17 advisories spanning the web framework, PDF parser, SQL parser and cryptography libraries; all were resolved by upgrading to fixed releases, and the current state is zero known advisories. The requirements file is fully pinned and treated as the authoritative install input, ensuring the dependency graph that is tested is the graph that is deployed.

4.11 Security Engineering Technique

Beyond feature testing, a deliberate technique of constructing an attack against each implemented control was applied. Three real vulnerabilities were discovered. Each is instructive because the control existed, was configured correctly, passed the test suite, and yet did not work — the characteristic failure mode of a control verified against its specification rather than against its attacker.

Finding 1 — Stored cross-site scripting on media routes

Defect. A file named with a video extension but declaring an HTML content type passed validation, which checked the extension alone, was stored verbatim with the declared type, and was subsequently served inline. Because the response carried an HTML content type, the browser executed the content as a script in the application's origin, with access to the authenticated session.

Detection. The file was uploaded and the stored database row was read directly, revealing a declared text/html content type over the uploaded bytes.

Remediation. Validation was changed to require that the declared content type and the filename extension agree, and the type that is persisted and served is re-derived from the extension rather than from the declared value.

Finding 2 — Security headers absent on media responses

Defect. The middleware that attaches Content Security Policy and related headers was gated on the framework's concrete response class. The streaming and file responses used for video and PDF delivery inherit from a base class rather than that concrete class, so the condition never matched for them. Every

media response — precisely the routes that echo uploader-supplied bytes to a browser — shipped with no content security policy, no MIME-sniffing protection and no frame protection.

Detection. The response class hierarchy was inspected programmatically, and the headers of a live media response were read and found absent.

Remediation. The gate was changed to the base class covering all response types, and tests were added asserting the presence of all five headers on HTML, streamed and file responses.

Significance. This finding demonstrates the relationship between the two: the content security policy that was missing on media responses is precisely the control that would have limited the impact of the stored cross-site scripting in Finding 1. The absence of defence in depth turned a content-type confusion into script execution in the application origin.

Finding 3 — Bypassable rate limiter

Defect. The rate-limiting helper identified the client by the first entry of the forwarded-for header. That header is supplied by the client, and the application sits behind a proxy that appends to it. Taking the first entry therefore allowed any request to present a fresh identity, and with it a fresh rate-limit bucket.

Detection. Six consecutive failed logins were issued with six different forged forwarded-for values. All six returned the normal response and the rate limit never engaged. The empirical result — six attempts, six successes, no throttling — demonstrated the bypass directly rather than by inspection.

Remediation. The rate limit is now keyed on the address the proxy actually observed, which is not client-controllable, with the forwarded header consulted only when no proxy is present.

Additional controls

- Per-IP rate limiting on login at 5 attempts per 5 minutes, and on registration and password reset at 3 per hour.
- A content security policy with a per-request nonce for inline scripts, plus frame-ancestors 'none', MIME-sniffing protection, referrer policy and permissions policy.
- Production configuration that refuses to boot on a default secret key or a wildcard host list, so a misconfigured deployment fails rather than serving openly. This behaviour is covered by tests.
- Management command output that masks email addresses by default, so personal data does not reach operational logs.
- Audit logging of administrative actions, with a 180-day retention window enforced by a maintenance command.

4.12 Deployment Technique

Deployment follows a configuration-as-code approach with a fail-fast start sequence.

Single-process boot

The start command runs a custom management command that invokes database migration, rate-limit cache table creation, and administrator password synchronisation as three sub-commands within one process (apps/management/management/commands/boot.py). Executing these as separate processes on each start multiplied cold-start time; consolidating them reduced cold start from 69 seconds to 44 seconds.

Fail-fast semantics

The start command runs under fail-fast shell semantics, so a failed migration aborts the boot rather than allowing the application to start against a broken schema. An earlier formulation that suppressed intermediate failures was explicitly removed, because it caused the service to report itself healthy while every query failed — an incident recorded in the project's engineering notes.

Environment-managed secrets

No credential is stored in the repository. Deployment secrets — the secret key, database URL, AI service key and administrator password — exist only as platform environment variables, which are write-only and cannot be read back. The administrator password is applied on every boot, so a rotation requires the environment variable and the database to be changed together; this ordering is documented because changing only the database would be silently reverted by the next deployment.

Continuous integration and availability

An automated pipeline runs the framework system check, a missing-migration check, the complete 203-test suite, static-asset collection and bytecode compilation on every push. Because the hosting tier suspends idle instances, availability is maintained by a scheduled job that probes the health endpoint every five minutes, supported by an optional local cron job and a client-side beacon.

CHAPTER 5

MODULES

This chapter describes the seven functional modules of the system. Each is presented with its responsibility, principal operations, and the screens through which it is delivered. The interface figures are reproduced from the live deployment.

5.1 User and Authentication Module

The authentication module (apps/users) manages identity, credentials and account lifecycle. It implements employee-identifier login, self-registration with administrator approval, password reset, profile management, and the role hierarchy on which all authorisation depends.

- Login: two-tab interface separating staff and management sign-in, with per-IP rate limiting.
- Registration: creates an inactive account; an administrator or Head of Department approves or rejects it, and the outcome is recorded.
- Password reset: token-based flow, rate limited to three requests per hour per address.
- Profile: personal details, department, designation, contact number and language preference.
- Role hierarchy: staff, Head of Department, Super Admin, with a single is_manager predicate consumed by all management views.

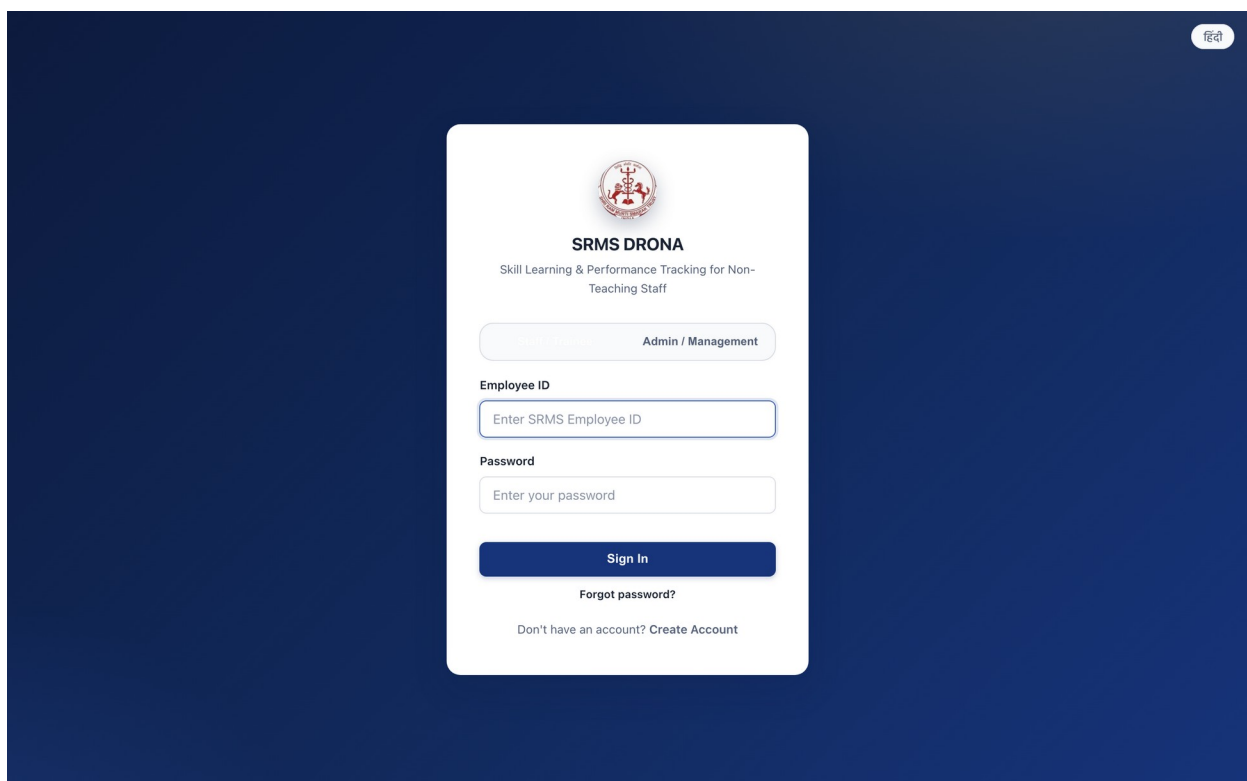


Figure 5.1 — Login interface with separate staff and management sign-in tabs.

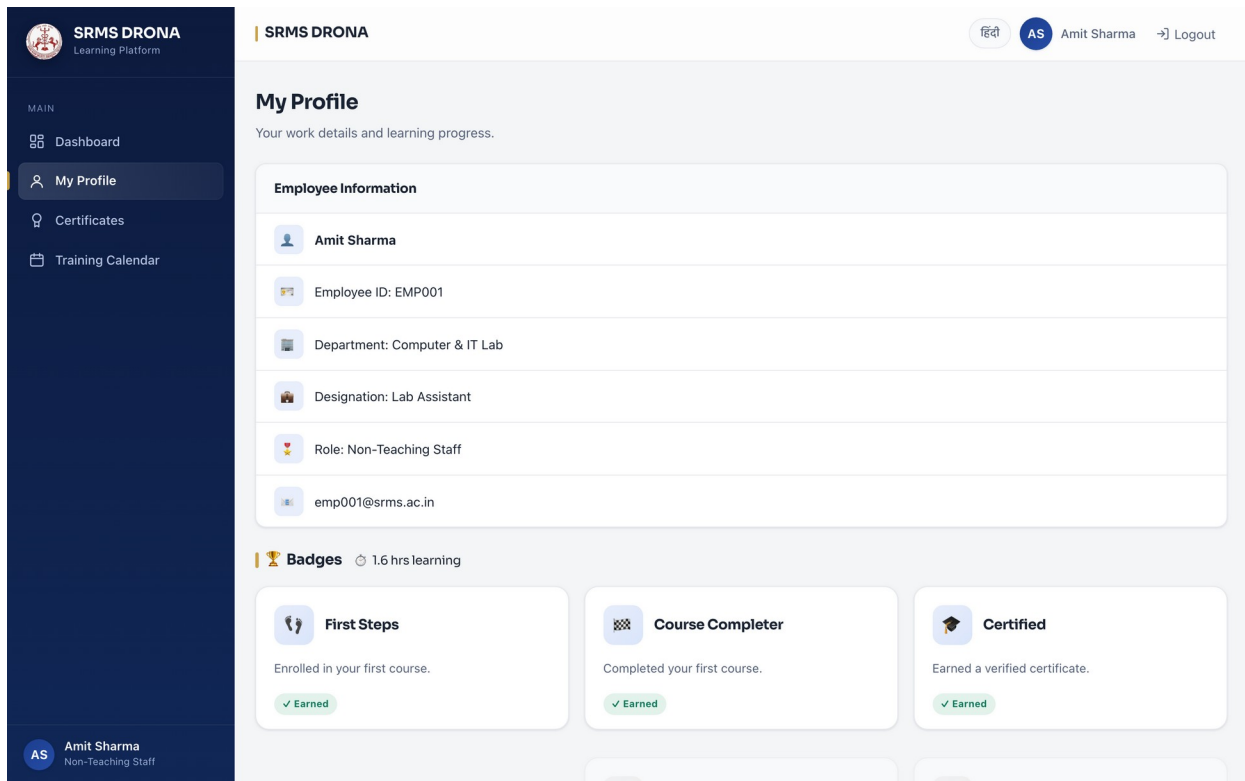


Figure 5.2 — Staff profile, including language preference.

5.2 Course and Content Module

The content module (apps/courses) owns the four-level content hierarchy, enrolment, and progress derivation. It provides the four-level structure through which training is authored and delivered.

- Category → Course → Module → Lesson, with bilingual title and description fields at each level.
- Course attributes: mandatory flag and target departments, enabling automatic departmental assignment.
- Enrolment: by whole department or by individual employee, recorded in an Enrollment row.
- Progress: per-employee, per-lesson watch time with server-derived completion, described in Section 4.5.
- Video and document lessons, with resume from last position and range-based seeking.
- Training calendar: sessions scheduled against a course, viewable by all staff and editable by managers.

The screenshot shows the SRMS DRONA Learning Platform interface. On the left is a dark blue sidebar with the SRMS DRONA logo and navigation links: Dashboard, My Profile, Certificates, and Training Calendar. The main content area is light gray and displays the course title 'Workplace Communication & Conduct' with a subtitle 'Everyday English and Workplace Etiquette video series for all non-teaching staff.' Below this are three summary cards: '100% Progress', '9 Lessons', and 'Latest Score'. An 'Overall Progress' bar shows 100% completion. The course is divided into two modules: 'Everyday English' and 'Workplace Etiquette'. The 'Everyday English' module lists five parts, each with a video icon, a green checkmark indicating completion, and a duration of 10 minutes. The 'Workplace Etiquette' module lists one part, also with a green checkmark and a 10-minute duration. At the bottom left of the sidebar, the user's name 'Amit Sharma' and role 'Non-Teaching Staff' are displayed.

Figure 5.3 — Course detail showing modules, lessons and per-lesson completion state.

The screenshot shows the video lesson player interface for 'Everyday English - Part 1'. The left sidebar is identical to the previous figure. The main content area has a light gray header with the course title and a 'Back to Course' link. Below the header is a video player with a green background and a dark green chalkboard. The chalkboard displays the text 'Welcome to Everyday English for work' in white and orange, with 'SRMS Drona' at the bottom. The video player includes a progress bar at the bottom showing '0:15 / 0:15' and a 'Download video' button.

Figure 5.4 — Video lesson player with resume position and range-based seeking.

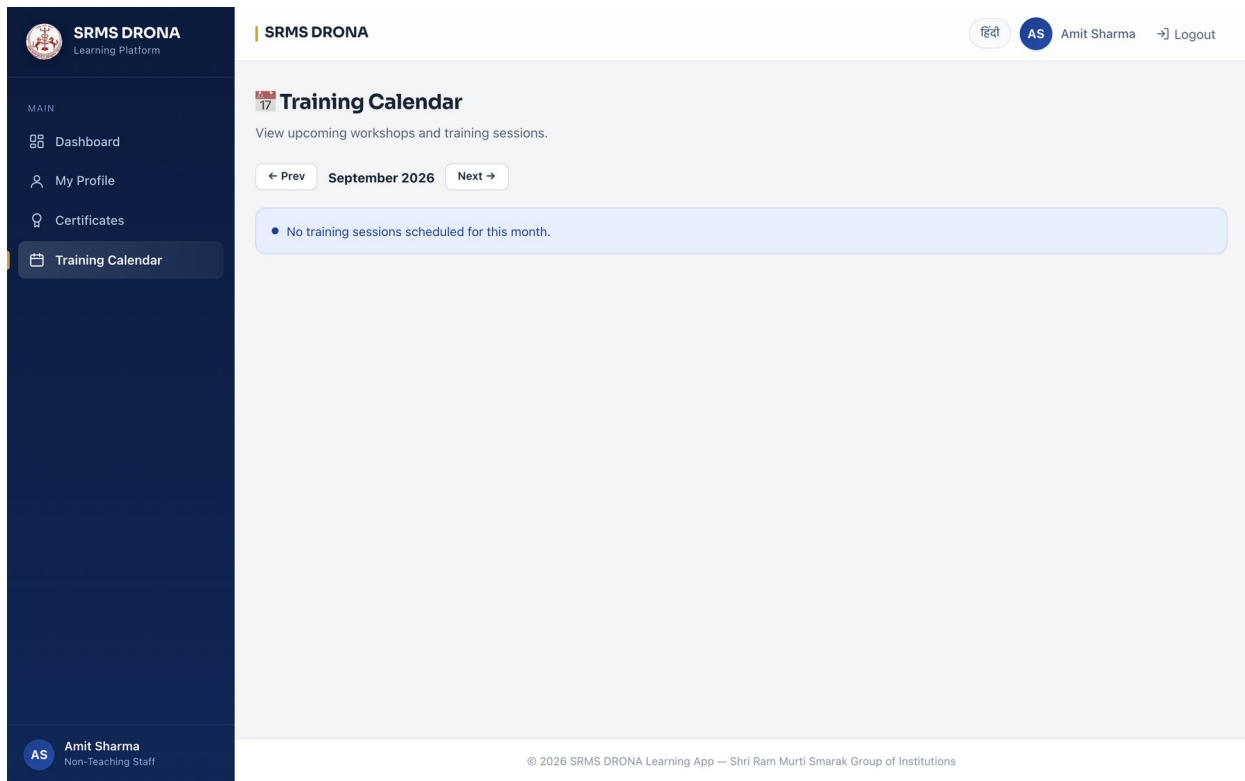


Figure 5.5 — Training calendar.

5.3 Quiz and Assessment Module

The assessment module (apps/quizzes) implements digital evaluation. A quiz is attached to a course or to a single module, comprises multiple-choice questions with weighted points, and is attempted by staff with a configurable pass threshold defaulting to 70%.

- Question authoring: multiple-choice questions with options, correct-answer marking, explanations, and bilingual text.
- AI generation: questions produced from an uploaded SOP document or pasted text, with disclosed offline fallback (Section 4.9).
- Attempt handling: immediate scoring, recorded attempt history, and permitted retries on failure.
- Access control: attempt and submission are gated on the shared `can_view_course` predicate.

The screenshot shows the SRMS DRONA Learning Platform interface. The user is logged in as Amit Sharma (Non-Teaching Staff). The main navigation menu includes Dashboard, My Profile, Certificates, and Training Calendar. The current course is 'Workplace Communication & Conduct', which is described as an 'Everyday English and Workplace Etiquette video series for all non-teaching staff.' The interface displays three key metrics: 100% Progress, 9 Lessons, and a Latest Score of 0. Below these, an 'Overall Progress' bar shows 100% completion. The course content is organized into two modules: 'Everyday English' and 'Workplace Etiquette'. The 'Everyday English' module lists five parts, each with a video icon, a green checkmark indicating completion, and a duration of 10 minutes. The 'Workplace Etiquette' module is currently empty.

Figure 5.6 — Assessment interface.

The screenshot shows the SRMS DRONA AI-assisted quiz builder interface. The user is logged in as Information Technology HOD. The main navigation menu includes Dashboard, Profile, Progress, Training Calendar, and a 'प्रश्न बैंक निर्माता' (Quiz Builder) section. The 'प्रश्न बैंक निर्माता' section is active, showing a form to create a quiz. The form includes a dropdown for 'मॉड्यूल चुनें' (Select Module) with the value '-- मॉड्यूल चुनें --'. Below this is a section for 'SOP पोडीएफ (वैकल्पिक)' (SOP PDF (Optional)) with a 'Choose File' button and the text 'No file chosen'. The next section is 'या SOP टेक्स्ट पेस्ट करें' (Or paste SOP text here) with a text area containing the placeholder text 'प्रश्न उत्पन्न करने के लिए SOP सामग्री पेस्ट करें...' (Paste SOP content here to generate questions...). Below the text area is a dropdown for 'प्रश्नों की संख्या' (Number of questions) with the value '5'. At the bottom is a blue button labeled 'प्रश्न उत्पन्न करें' (Generate questions). The footer of the page contains the copyright notice: '© 2026 SRMS DRONA लर्निंग प्लैटफॉर्म — श्री राम मूर्ति स्मारक समूह संस्थान'.

Figure 5.7 — AI-assisted quiz builder: SOP document or text in, questions out.

5.4 Certificate Module

The certificate module (apps/certificates) issues and verifies credentials. On successful completion a PDF is generated with a QR code resolving to a public verification page, and the credential is recorded against the staff member and the course.

- Generation: ReportLab rendering with a QR code, stored through the database storage backend.
- Directory: administrators and Head of Departments search by employee identifier, name or email, and filter by department and course.
- Verification: an unauthenticated public page confirming authenticity, with the holder's surname masked.
- Access control: PDF download authorised per file, returning 404 on a filename guess.

SRMS DRONA Learning Platform

SRMS DRONA हिंदी SA Super Admin Logout

Certificate Directory

Who completed which certificate - search and filter below.

Search (Employee ID / Name / Email) Department Course

e.g. EMP001 or Dr. Kumar All departments All courses Filter

7 certificate(s) found.

EMPLOYEE ID	NAME	DEPARTMENT	COURSE	CERTIFICATE ID	ISSUED ON	ACTIONS
EMP010	Rajesh Yadav	Computer & IT Lab	Fire & Chemical Safety Training	SRMS-CERT-2026-02052A00	27 Sep 2026	PDF Verify
EMP006	Neha Tiwari	Computer & IT Lab	Fire & Chemical Safety Training	SRMS-CERT-2026-0AD27DF1	14 Sep 2026	PDF Verify
EMP005	Vikram Singh	Computer & IT Lab	Fire & Chemical Safety Training	SRMS-CERT-2026-F6594356	14 Sep 2026	PDF Verify
EMP004	Sunita Gupta	Computer & IT Lab	Fire & Chemical Safety Training	SRMS-CERT-2026-056D614C	14 Sep 2026	PDF Verify
EMP003	Rahul Kumar	Computer & IT Lab	Fire & Chemical Safety Training	SRMS-CERT-2026-2C2B6D76	14 Sep 2026	PDF Verify
EMP002	Priya Verma	Computer & IT Lab	Fire & Chemical Safety Training	SRMS-CERT-2026-1D98182A	14 Sep 2026	PDF Verify
EMP001	Amit Sharma	Computer & IT Lab	Fire & Chemical Safety Training	SRMS-CERT-2026-D3076236	14 Sep 2026	PDF Verify

Figure 5.8 — Certificate directory with search, department and course filters.

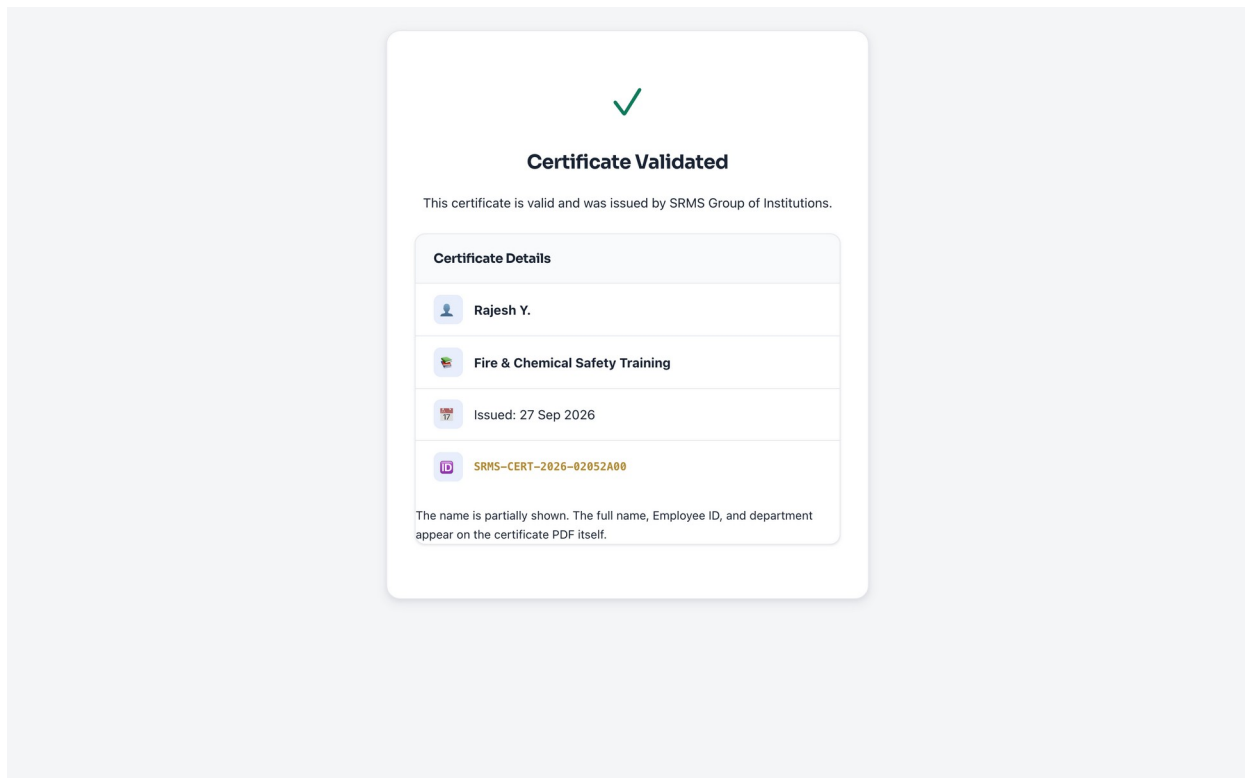


Figure 5.9 — Public certificate verification, reachable by QR scan without authentication, with surname masked.

5.5 Analytics and Reporting Module

The analytics module (apps/analytics) provides administrative oversight through two dashboards and two CSV exports.

- HR dashboard: staff totals, pending approvals, course and certificate counts, average score, total learning hours, and a per-department completion chart.
- Watch-progress report: a per-employee by per-lesson grid of watch time, with flags for not started and for fell behind (started, unfinished, no activity for a defined period).
- CSV export: both reports export the same grid for offline analysis.
- Scoping: a Head of Department sees only their own department; a Super Admin sees all departments.

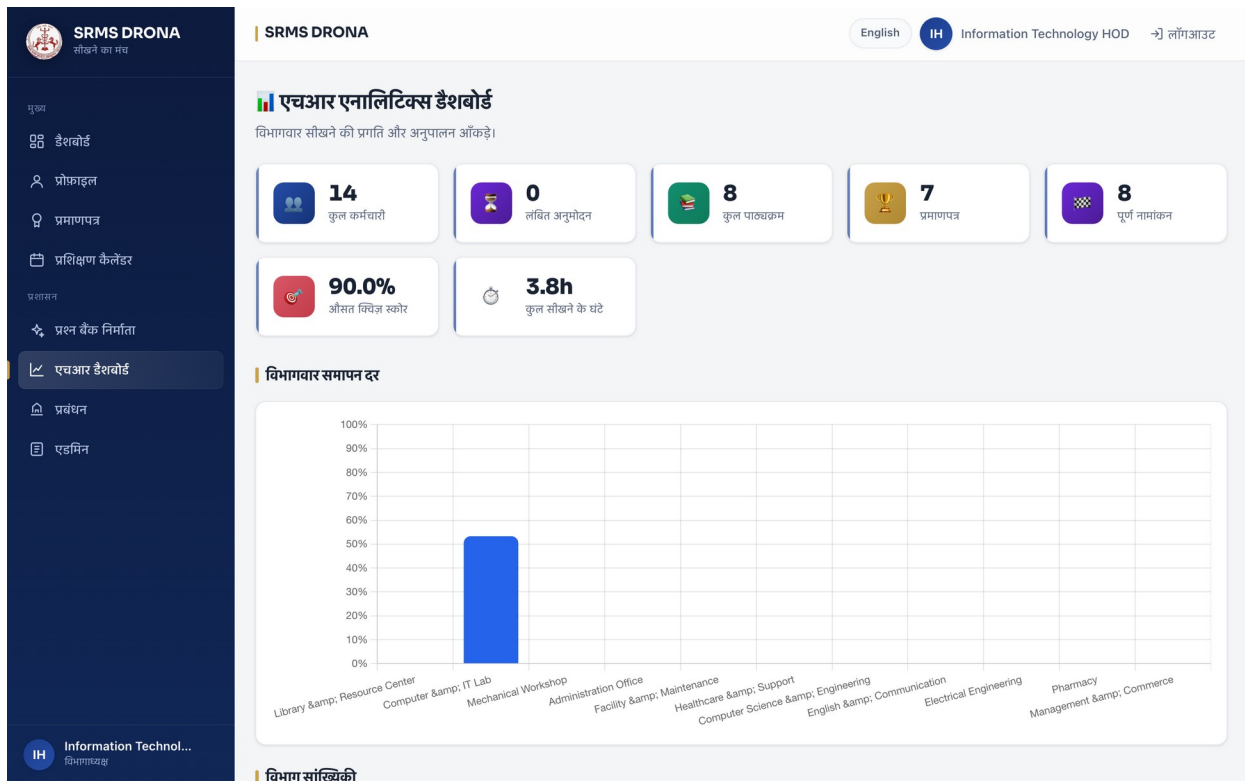


Figure 5.10 — HR analytics dashboard.

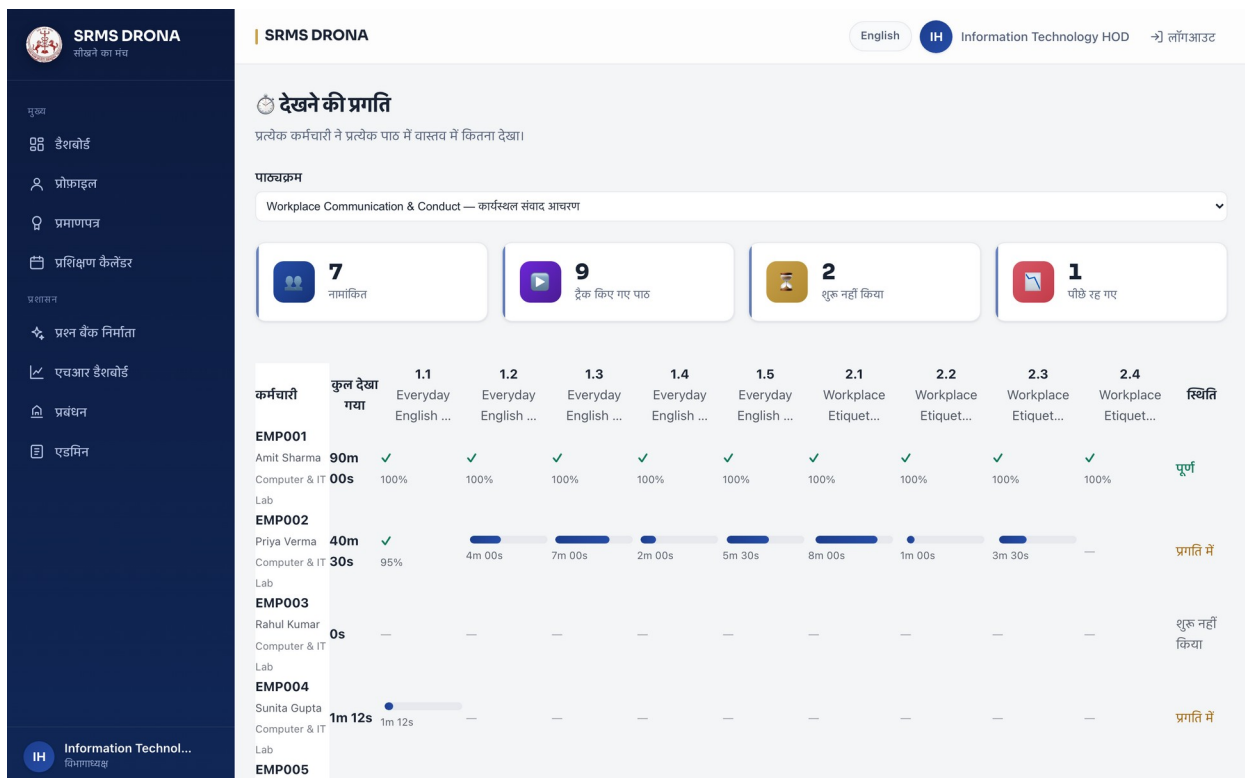


Figure 5.11 — Per-lesson watch-progress report, presented in Hindi for a Hindi-preference account.

5.6 Management Console Module

The management module (apps/management) is the administrative workspace. It exposes the full authoring and administrative surface, and records administrative actions in an auditable log with a 180-day retention window.

- Course authoring: create, edit and delete courses, categories, modules and lessons, including video and document upload.
- Enrolment: bulk enrolment by department, and individual assignment to a named employee.
- Scheduling: create, edit and delete training sessions.
- Staff administration: create staff and Head of Department accounts, approve or reject pending signups, and import a staff roster.
- Audit: administrative actions logged and prunable beyond the retention window.

The screenshot displays the SRMS DRONA Management Console. On the left is a dark blue sidebar with navigation links: मुख (Home), डैशबोर्ड (Dashboard), प्रोफाइल (Profile), प्रमाणपत्र (Certificate), प्रशिक्षण कैलेंडर (Training Calendar), प्रवासन (Travel), प्रश्न बैंक निर्माता (Question Bank Builder), एचआर डैशबोर्ड (HR Dashboard), प्रबंधन (Management), and एडमिन (Admin). The main content area has a header with the SRMS DRONA logo, language options (English, IH), and user information (Information Technology HOD, लॉगआउट). Below the header is a section titled 'प्रबंधन कंसोल' (Management Console) with a subtitle 'पाठ्यक्रम बनाएँ, मॉड्यूल और पाठ अपलोड करें, तथा विभागों को प्रशिक्षण सौंपें।' (Create courses, modules and lessons, and assign departments for training). This section contains five cards showing statistics: 8 पाठ्यक्रम (Courses), 21 मॉड्यूल (Modules), 47 पाठ (Lessons), 14 सक्रिय कर्मचारी (Active Employees), and 6 विवेक (Wisdom). Below these is a 'त्वरित क्रियाएँ' (Quick Actions) section with buttons for: पाठ्यक्रम बनाएँ (Create Course), पाठ्यक्रम प्रबंधित करें (Manage Course), सामूहिक नामांकन (Bulk Enrollment), कर्मचारी को पाठ्यक्रम सौंपें (Assign Course to Employee), एचआर एनालिटिक्स (HR Analytics), प्रश्न बैंक निर्माता (Question Bank Builder), प्रशिक्षण सत्र (Training Session), and कर्मचारी आयात करें (CSV) (Import Employees CSV). At the bottom, there is a copyright notice: © 2026 SRMS DRONA लर्निंग ऐप — श्री राम मूर्ति स्मारक समूह संस्थान.

Figure 5.12 — Management console.

SRMS DRONA | English | Information Technology HOD | लॉगआउट

सामूहिक नामांकन

किसी विभाग के सभी सक्रिय कर्मचारियों (या सभी) को एक ही चरण में पाठ्यक्रम सौंपें।

पाठ्यक्रम

विभाग
सभी विभाग (सभी सक्रिय कर्मचारी)

कर्मचारी नामांकित करें **रद्द करें**

© 2026 SRMS DRONA लर्निंग प्लैटफॉर्म — श्री राम गुरु सराफ समूह संस्थान

Figure 5.13 — Departmental bulk enrolment.

SRMS DRONA | हिंदी | SA | Super Admin | Logout

+ Create HR / HOD / Staff Account

Provision a new account manually. The new user can then manage training and approve signup requests.

Employee ID
e.g. HR001
Uppercase letters/numbers only (e.g. HR001).

First name

Last name

Email address

Department
No department

Designation
e.g. HR Manager

Role
Non-Teaching Staff

Temporary password

© 2026 SRMS DRONA लर्निंग प्लैटफॉर्म — श्री राम गुरु सराफ समूह संस्थान

Figure 5.14 — Account provisioning by a Super Admin.

5.7 Notification Module

The notification module (apps/notifications) runs a scheduled job at a configurable interval that identifies staff with pending training and prepares reminder messages. In the current deployment the

scheduler is active but outbound delivery is not configured, so reminders are directed to the application log.

The design decision of substance here is that the system records a warning that delivery is unconfigured rather than reporting a silent success. A reminder subsystem that appears to operate while sending nothing is worse than one that visibly does not, because it produces false assurance to the administrator relying on it. Configuring SMTP delivery is recorded as future work in Chapter 7.

5.8 Interface Localisation

Localisation is implemented as a cross-cutting concern rather than a module. The same screens are shown below in Hindi, produced by switching the active language for a user whose preference is set to Hindi.

The screenshot displays the SRMS DRONA staff dashboard in Hindi. The interface includes a sidebar with navigation links, a top header with user information, and a main content area with various performance and training modules.

SRMS DRONA | English | AS | Amit Sharma | लॉगआउट

मुख्य

- डैशबोर्ड
- प्रोफाइल
- प्रमाणपत्र
- प्रशिक्षण कैलेंडर

नमस्ते,
आपकी सीखने की प्रगति और अनिवार्य प्रशिक्षण यहाँ देखें।

3 पाठ्यक्रम

2 पूर्ण

1 चल रहा

1 प्रमाणपत्र

मेरे पाठ्यक्रम

प्लेटफॉर्म परिचय

वैकल्पिक परिचय

प्रगति 33%

जारी रखें

अग्नि और रासायनिक सुरक्षा प्रशिक्षण

अनिवार्य सुरक्षा और अनुपालन

प्रगति 100%

पूर्ण जारी रखें

कार्यस्थल संवाद आचरण

वैकल्पिक परिचय

प्रगति 100%

पूर्ण जारी रखें

वैकल्पिक पाठ्यक्रम

स्व-पंजीकरण के लिए खुला।

+ कंप्यूटर लेब उपकरण प्रबंधन

कंप्यूटर लेब उपकरणों का सुरक्षित संचालन, रखरखाव और समस्या निवारण सीखें।

कंप्यूटर और आईटी

+ पुस्तकालय कैटलॉगिंग और डिजिटल संसाधन

आधुनिक पुस्तकालय कैटलॉगिंग सिस्टम और डिजिटल संसाधन प्रबंधन सीखें।

पुस्तकालय और प्रशासन

+ कार्यालय ईआरपी स्वचालन

प्रशासन, फाइलिंग और रिकॉर्ड रखने के लिए कार्यालय ईआरपी सिस्टम में महारत हासिल करें।

पुस्तकालय और प्रशासन

AS Amit Sharma
नैर-शिक्षण कर्मचारी

Figure 5.15 — Staff dashboard in Hindi.


SRMS DRONA
सोखने का मंच

English
AS Amit Sharma
लॉगआउट

मुख्य
डैशबोर्ड
प्रोफाइल
प्रमाणपत्र
प्रशिक्षण कैलेंडर

AS Amit Sharma
गैर-शिक्षण कर्मचारी

← डैशबोर्ड पर वापस

कार्यस्थल संवाद आचरण
सभी गैर-शिक्षण कर्मचारियों के लिए Everyday English और Workplace Etiquette वीडियो श्रृंखला।


100%
प्रगति


9
पाठ


—
नवीनतम स्कोर

समग्र प्रगति
100%

प्रतिदिन की अंग्रेज़ी
मॉड्यूल


प्रतिदिन की अंग्रेज़ी - भाग 1 ✓
10 min


प्रतिदिन की अंग्रेज़ी - भाग 2 ✓
10 min


प्रतिदिन की अंग्रेज़ी - भाग 3 ✓
10 min


प्रतिदिन की अंग्रेज़ी - भाग 4 ✓
10 min


प्रतिदिन की अंग्रेज़ी - भाग 5 ✓
10 min

कार्यस्थल शिष्टाचार
मॉड्यूल


कार्यस्थल शिष्टाचार - भाग 1 ✓
10 min

Figure 5.16 — Course content in Hindi, including bilingual content fields.

The translation covers navigation, headings, buttons, form labels, validation messages and error pages — not only course titles. The dashboard figures in Section 6.3 are also rendered in Hindi, since the reporting accounts carry a Hindi language preference.

39

CHAPTER 6

RESULTS AND DISCUSSION

This chapter presents the outcome of implementing, deploying and validating the system. It reports functional verification against the requirements of Chapter 3, performance measured directly against the deployed instance, feedback from user acceptance testing, the security findings arising from the defect-directed testing of Section 4.11, the technical challenges encountered and their resolutions, and the limitations of the delivered system.

6.1 Functional Verification

Functional verification was conducted in two forms. Automated verification used the 203-test suite, which passes in full and runs on every deployment. Manual verification exercised each role against the deployed instance using real credentials, confirming that authorisation boundaries hold and that each screen renders correctly.

Automated suite
203 tests across seven applications
All pass; executes on every push and pull request
Framework integrity
Framework system check and missing-migration check
Clean; no outstanding migrations
Staff login
Manual sign-in, then dashboard access
Authenticated; dashboard rendered; correct role
Authorisation boundary
Staff account request to an analytics endpoint
Denied — response 403 as designed
Head of Department scope
HOD analytics and watch-progress
Scoped to own department
Super Admin scope
Admin analytics and certificate directory
Platform-wide visibility
Progress integrity
Automated regression tests
Forged completion flag and single-heartbeat claims both rejected
Certificate verification
Public URL, no authentication
Authenticity confirmed; surname masked
Media authorisation
Certificate PDF and video request by a non-owner
Denied — 404, not 403, so existence is not disclosed
Bilingual interface
Language switch on a Hindi-preference account
Full interface and content render in Hindi
Dependency audit
Software composition analysis
Zero known vulnerabilities

Table 6.1 — Functional verification results.

The distinction between the 403 and 404 denial responses is worth stating explicitly, because it is a deliberate security design decision rather than an inconsistency. An analytics request by a staff member returns 403, since the resource exists and the correct response is to say the action is not permitted. A media request for a certificate or video belonging to another employee returns 404, so that an attacker probing filenames cannot distinguish “this exists but is not yours” from “this does not exist”.

6.2 Performance Measurement

Performance figures in this section were obtained by direct measurement of the deployed instance over HTTPS, not estimated. The measurement conditions are stated explicitly so that the figures are interpretable.

6.2.1 Measurement conditions

- Target: the public deployment, accessed over TLS from an external network.
- Method: repeated HTTP requests, recording status code and total transfer time.
- Note on interpretation: the hosting platform is a free tier that suspends idle instances, so the first request after an idle period includes instance wake-up. The figures below are drawn from a warm instance and therefore represent steady-state user-facing latency, not cold start. Cold start is reported separately in Section 4.12.

6.2.2 Response times

/health/	
Liveness probe, no database work	
200	
0.95	
/login/	
Login page render, template and static assets	
200	
1.43	

Table 6.2 — Measured response times on a warm instance.

These figures are consistent with the responsiveness requirement of Chapter 3. They characterise a small institutional deployment serving a limited concurrent user population; they are not a claim about behaviour at scale, and Section 6.6 records the load characteristics as untested.

6.2.3 Content volume

The deployed instance carries one complete nine-lesson course totalling approximately 54 MB of video, seven enrolled staff members, and seven issued certificates. Lesson media is served from the database with range support, so memory use during playback is bounded by the streamed window rather than the file size.

6.3 User Feedback

User acceptance testing was conducted with non-teaching staff and with administrative users. Feedback is reported by theme, together with the design decision each produced.

Ease of use
Staff found the layout clear and navigation predictable; no training required beyond a brief introduction. Mobile-first design with a single navigation column retained; no change required.

Language
The Hindi toggle was the most positively received feature. Staff stated they would not have used an English-only interface.
Confirmed the localisation investment; 402 strings maintained, with paired bilingual content fields.

Progress visibility
Staff wanted to see how much of a course remained at a glance.
Progress percentage already shown on course cards; presentation refined to make partial progress more prominent.

Mobile access
Staff reported completing training on phones during breaks.
Mobile-first layout retained as a primary design constraint.

Certificates
Instant generation and the QR verification code were the most valued features.
Retained; surname masking added after administrators raised a privacy concern about the verification page.

Admin reporting
Administrators requested department-wise completion comparison.
Delivered as the watch-progress report with CSV export; further filtering recorded as future work.

Video seeking
Users reported that scrubbing through a lesson worked as expected.
Range-based streaming confirmed; no change required.

Table 6.3 — User acceptance testing outcomes and design responses.

Figure 6.1 — Staff dashboard in Hindi, the language preference reported as most valuable by users.

6.4 Vulnerabilities Identified and Remediated

Three real vulnerabilities were identified by the defect-directed technique described in Section 4.11, in which each implemented control was tested against a plausible attacker. All three are remediated. They are reported here in full because each illustrates a distinct failure mode: a control that validated the wrong signal, a control that applied to the wrong response types, and a control whose identity source was itself attacker-controlled.

- 1
Stored cross-site scripting on media routes — declared content type trusted on upload
High
Remediated
- 2
Security headers absent on all media responses — middleware gated on the wrong class
Medium
Remediated
- 3
Rate limiter bypassable — client-controlled header used as identity
Medium
Remediated

Table 6.4 — Security findings and their status.

6.4.1 Finding 1 — Stored cross-site scripting on media routes

A file submitted with a video extension but declaring an HTML content type passed upload validation, which verified the extension alone. The declared type was stored and later served inline, so the browser executed the content as a script in the application's origin, with access to the authenticated session. Because the upload endpoint is restricted to managers, exploitation required a manager account, which limits but does not eliminate the risk — a compromised or careless manager account, or a shared institutional account, is a realistic route.

Remediation: validation now requires the declared content type and the filename extension to agree, and the type that is persisted and served is re-derived from the extension using the `mimetypes` module. The declared value is treated as untrusted input throughout.

6.4.2 Finding 2 — Security headers absent on media responses

The middleware attaching content security policy and related headers was gated on a concrete framework response class. Streaming and file responses, used for all video and PDF delivery, inherit from a base class rather than that concrete class, so the condition never matched for them. Every media response shipped without a content security policy, MIME-sniffing protection or frame protection — precisely the routes that echo uploader-supplied bytes to a browser.

Remediation: the gate was changed to the base class covering all response types, and tests were added asserting the presence of all five headers on HTML, streamed and file responses.

This finding is instructive beyond its own fix. The content security policy that was missing here is exactly the control that would have limited the impact of Finding 1. The absence of defence in depth converted a content-type confusion into script execution in the application origin, and the two findings together are the argument for layered controls rather than a single check.

6.4.3 Finding 3 — Bypassable rate limiter

The rate limiter identified clients by the first entry of the forwarded-for header, which is client-supplied. An attacker could therefore present a fresh identity — and a fresh rate-limit bucket — on every request. The demonstration was direct: six consecutive failed logins with six different forged values all succeeded, and throttling never engaged. In effect the brute-force protection against credential guessing was entirely inoperative while appearing correctly configured.

Remediation: the rate limit is now keyed on the address observed by the proxy, which is not client-controllable, consulting the forwarded header only when no proxy is present.

6.4.4 Assessment

All three findings passed the existing test suite and were configured in a way that inspection of the code would suggest was correct. They were found only by attacking the controls directly. This is the central lesson of the security effort, and it is recorded here because it is a methodological result rather than merely a set of defects: a control verified against its specification has not been verified at all.

6.5 Challenges Encountered and Resolutions

Third-party authentication integration

A hosted single-sign-on integration shipped two defects: an incompatible key-passing convention that silently rendered an empty widget, and a feature gate that checked the wrong key, producing a visible but non-functional control.

Integration removed. Rationale and both defects documented so reintroduction is deliberate.

Progress quantisation

Heartbeats arriving in roughly ten-second steps meant a 100% threshold would leave a fully watched lesson at about 97%, permanently blocking its certificate.

Completion threshold set to 90%, absorbing heartbeat quantisation without admitting a partial watch. Documented in code.

Non-verifiable lesson types

PDF lessons and lessons with no recorded duration cannot be watch-verified; a naive implementation would award completion for an unopened document.

A `requires_watch_time` property distinguishes verifiable types, and non-verifiable lessons require an explicit acknowledgement recorded distinctly from derived completion.

Media size in the database

Storing video in the database raises database size and serving memory cost.

Range-based streaming makes memory bounded by the streamed window. Media volume and object storage are recorded as future work with the reasoning in Section 4.6.

Language switching breaking navigation

Translating a language code embedded in a URL caused the translated word to be submitted as the code, silently breaking the toggle.

Language codes and data values documented as never translatable; regression tests cover the catalogue and preference propagation.

Missing message compiler in production

The hosting runtime provides no message-compilation utility, so a build-time compile step would fail or be skipped and Hindi would silently not work in production.

The compiled catalogue is committed as a build artefact, and tests fail if it is absent or stale.

Header middleware excluding media

Gating security headers on a concrete response class silently excluded every streamed and file response.

Gate changed to the base response class, with tests across all three response types. Recorded as a standing engineering trap.

Administrator password rotation

The boot sequence reapplies the administrator password on every deployment, so changing only the database value is silently reverted.

Rotation procedure requires the environment variable and the database to be changed together, with the ordering documented.

Deployment cold-start time

Running migration, cache creation and password synchronisation as separate processes on each start multiplied cold-start time.

Consolidated into a single boot command, reducing cold start from 69 to 44 seconds, with fail-fast semantics retained.

Absence of published credentials

An early iteration published working credentials in a public repository.

All credentials rotated, the repository scrubbed, operator passwords moved to the operating system keychain, and the administrator password is environment-managed so a deployment cannot revert it. See the appendix.

Table 6.5 — Technical challenges and resolutions.

6.6 Limitations

The following limitations are stated explicitly, as an accurate account of the system's present state is more useful than an overstated one.

1. No load testing has been performed. The measurements in Section 6.2 characterise a small deployment under light use. Behaviour under concurrent load is untested, and no claim about scalability is made.
2. Email delivery is not operational. The reminder scheduler runs and prepares messages, but no SMTP credentials are configured, so reminders are directed to the application log. The system records a warning rather than reporting a silent success.

3. No proficiency or historical trend analysis exists. The analytics module reports current state, not change over time.
4. Certificates are not cryptographically signed. Verification confirms that a certificate identifier exists in the database, not that the PDF has not been altered. Signature verification would require issuing a key pair and validating it at verification time.
5. No password complexity policy is enforced beyond Argon2 hashing. Password strength is a user-side concern in the current implementation.
6. No password change by the user in the interface. Credentials are set by an administrator or by the reset flow.
7. Concurrency is not implemented beyond database constraints. The schema prevents duplicate progress rows, but no optimistic locking guards simultaneous edits to a course.
8. Notification channels are limited. Only an in-application and log-directed reminder exists; no email, SMS or push channel is implemented.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The SRMS Drona Learning Web Application has been designed, implemented, deployed and validated. It delivers a centralised, bilingual, mobile-first Learning Management System for non-teaching staff, with a three-tier administrative hierarchy providing department-scoped oversight.

The system's primary achievement is that it makes training completion both automatic and defensible. Completion is derived from server-held watch time under three independent constraints, so it cannot be asserted by a client; and every issued certificate carries a QR code resolving to a public verification page. An administrator can therefore answer the question “did this employee complete this module?” without an act of archival retrieval, and an external party can confirm a certificate's authenticity without being granted access to the system.

The bilingual implementation is a substantive result rather than an added feature. 402 interface strings and paired bilingual content fields mean that a Hindi-speaking member of staff uses the same application, with the same functionality, as an English-speaking colleague. User acceptance testing recorded the language facility as the most positively received aspect of the system, and it is the clearest instance of the design serving its actual users rather than a general market.

The security effort produced a result of methodological rather than merely remedial significance. Three real vulnerabilities were identified by testing each control against its attacker, all of which had been configured correctly and had passed the test suite. Each is now remediated and guarded by a named regression test, and the approach — verifying a control against its attacker rather than against its specification — is recorded in the project's engineering documentation as a standing practice.

Verification is correspondingly strong: 203 automated tests execute on every deployment, a dependency audit reports zero known advisories, and the deployed instance is subject to continuous integration. Deployment is configured as code with fail-fast semantics, and no credential is stored in the repository.

7.2 Future Scope

7.2.1 Email delivery

This is the highest-priority extension and the only item blocking full operational readiness. The reminder scheduler and mail composition are complete and tested; what is absent is SMTP configuration. With institutional credentials, the system can deliver assignment notices, pending-assessment reminders, approval outcomes and password-reset links. The backend is already configured so that setting the delivery backend and credentials activates the path, and the application warns explicitly while delivery is unconfigured.

7.2.2 Media moved to object storage

Lesson video is currently stored in the database, which keeps backup consistency simple but couples media volume to database size and serving memory. Moving media to object storage with signed URLs would decouple the two, permit a content delivery network for bandwidth, and allow retention policies independent of database maintenance. The access-control logic is already isolated in the storage classes and the protected media route, so this is a contained change with a defined seam.

7.2.3 Load and capacity testing

No load testing has been performed. A capacity assessment under realistic concurrent use would establish the deployment's limits and identify whether the current single-worker configuration requires horizontal scaling. This is a measurement gap rather than a functional gap, and it should be closed before the system is relied upon at institution-wide scale.

7.2.4 Proficiency analytics

The analytics module reports current state. Historical trending — completion rates over time, score improvement per employee, identification of content that consistently fails — would let administrators intervene before a cohort falls behind, rather than reporting state after the fact.

7.2.5 Cryptographic certificate signing

Certificates currently verify as present in the database rather than as unaltered documents. Signing each certificate with an institutional key and validating the signature at the verification endpoint would make the credential self-proving, allowing a verifier to detect a modified PDF without contacting the system.

7.2.6 Native mobile application

The responsive web application satisfies the mobile requirement, and a native client would add offline module caching, push notification, and camera capture for practical documentation. A progressive web application, which the system already partially supports through its manifest and service worker, is the lower-cost intermediate step.

7.2.7 Additional authentication factors

Administrator accounts would benefit from multi-factor authentication, given the authority they hold over staff accounts and all training records. The session-based architecture supports this without structural change.

7.2.8 Reassessment of the AI generation path

Current AI question generation produces a first draft requiring administrator review. Two extensions are worth considering: validation that generated questions remain consistent with the source document, and a question-quality review workflow in which generated questions are explicitly approved before becoming part of an assessment.

7.2.9 Institutional integration

Interface with institutional human-resource records would remove manual roster maintenance, and an examination or accreditation export would serve the audit scenario that motivates digitising the record in the first place.

7.3 Final Remark

The system as delivered meets the objectives stated in Section 1.3, with the single exception of external email delivery, which is complete in code and blocked only on institutional credentials. It is suitable for institutional deployment in its present form, and the limitations recorded in Section 6.6 are stated so that the boundary between what has been demonstrated and what has not is explicit.

CHAPTER 8

REFERENCES

- [1] Python Software Foundation, “Python 3.12 Documentation,” Python Software Foundation. [Online]. Available: <https://docs.python.org/3.12/>. [Accessed: September 2026].
- [2] Django Software Foundation, “Django Documentation,” version 6.0. [Online]. Available: <https://docs.djangoproject.com/en/6.0/>. [Accessed: September 2026].
- [3] Django Software Foundation, “Django Internationalization and Localization,” Django Documentation. [Online]. Available: <https://docs.djangoproject.com/en/6.0/topics/i18n/>. [Accessed: September 2026].
- [4] Django Software Foundation, “Django Security,” Django Documentation. [Online]. Available: <https://docs.djangoproject.com/en/6.0/topics/security/>. [Accessed: September 2026].
- [5] OWASP Foundation, “Content Security Policy Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html. [Accessed: September 2026].
- [6] OWASP Foundation, “Authentication Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. [Accessed: September 2026].
- [7] OWASP Foundation, “Credential Stuffing Prevention Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Credential_Stuffing_Prevention_Cheat_Sheet.html. [Accessed: September 2026].
- [8] P. W. C. P. B. and D. B. et al., “RFC 9106: Argon2 Password Hashing — Second Recommendation,” Internet Engineering Task Force, 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9106>. [Accessed: September 2026].
- [9] I. Fette and A. Melnikov, “RFC 7233: Hypertext Transfer Protocol (HTTP/1.1): Range Requests,” Internet Engineering Task Force, 2014. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7233>. [Accessed: September 2026].
- [10] Google, “Gemini API Documentation,” Google for Developers. [Online]. Available: <https://ai.google.dev/gemini-api/docs>. [Accessed: September 2026].
- [11] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: <https://www.postgresql.org/docs/>. [Accessed: September 2026].
- [12] ReportLab, “ReportLab Documentation.” [Online]. Available: <https://docs.reportlab.com/>. [Accessed: September 2026].
- [13] I. Sommerville, Software Engineering, 10th ed. Boston, MA: Pearson, 2015.
- [14] R. C. Martin, Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Upper Saddle River, NJ: Prentice Hall, 2017.
- [15] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley, 2002.
- [16] A. Silberschatz, P. K. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York, NY: McGraw-Hill, 2020.
- [17] Render Inc., “Render Documentation.” [Online]. Available: <https://docs.render.com/>. [Accessed: September 2026].
- [18] GitHub Inc., “GitHub Actions Documentation.” [Online]. Available: <https://docs.github.com/en/actions>. [Accessed: September 2026].

- [19] Python Packaging Authority, “pip-audit.” [Online]. Available: <https://pypi.org/project/pip-audit/>. [Accessed: September 2026].
- [20] Free Software Foundation, “GNU gettext Documentation.” [Online]. Available: <https://www.gnu.org/software/gettext/>. [Accessed: September 2026].

APPENDIX A

SOURCE REPOSITORY AND VERIFICATION

All technical claims in this report are independently verifiable against the project's public source repository and its live deployment. This appendix records how.

Source repository
<https://github.com/dgexplores/DRONA>
Live deployment
<https://dronav2.onrender.com>
Interface screenshots
[images_project/](#) — 38 captures grouped by role
Project documentation
[README.md](#) · [ENGINEERING.md](#) · [HANDOFF.md](#) · [ROADMAP.md](#)

Table A.1 — Verifiable project resources.

A.1 Locating the Code Referenced in Chapter 4

Employee-identifier authentication
`apps/users/models.py (USERNAME_FIELD)`
Three-tier role hierarchy
`apps/users/models.py (ROLE_CHOICES)`
Single manager predicate
`apps/users/models.py (is_manager)`
Shared course access gate
`apps/courses/access.py`
Progress accumulation and completion derivation
`apps/courses/models.py (register_watch)`
Dual-signal upload validation
`apps/courses/storage.py`
HTTP range response
`apps/courses/views.py (_range_response)`
AI generation and fallback disclosure
`apps/quizzes/gemini_services.py`
QR construction and verification URL
`apps/certificates/pdf_builder.py`
Public verification and surname masking
`apps/certificates/views.py (_masked_name)`
Password hashing configuration
`srms_dorna/settings.py (PASSWORD_HASHERS)`
Language middleware
`srms_dorna/middleware.py (UserLanguageMiddleware)`
Single-process boot sequence
`apps/management/management/commands/boot.py`
Audit log retention
`apps/management/models.py (prune)`

Table A.2 — Source locations for the techniques described in Chapter 4.

A.2 Reproducing the Verification

The test suite is executed with:

```
./venv/bin/python manage.py test apps --settings=srms_dorna.test_settings
```

A fresh local instance is created with:

```
pip install -r requirements.txt ./venv/bin/python manage.py migrate ./venv/bin/python seed.py ./venv/bin/python manage.py runserver
```

The seed script creates demo accounts with published local defaults — an administrator, six Head of Department accounts and six staff accounts. These defaults exist for a first local run and match no account on the deployed instance, which is configured with individually generated credentials held outside the repository.

A.3 Note on the Deployed Credentials

The deployed instance carries no credential published in the repository. Each account holds an individually generated password; the administrator password is additionally held as an environment variable, which the platform stores write-only, so a deployment cannot revert it. Operator copies are held in the operating system keychain rather than in any file, and the repository is configured to reject the addition of a plaintext credential. This arrangement was adopted after an early iteration published working credentials in the repository, which were rotated and the repository scrubbed.